Project 3: Modeling and simulation of multi-drone swarms

Please use a new page for each part of a problem. That will help your GSI and grader!

Introduction

In this project, you will use a genetic algorithm to find optimal control parameters for a swarm of autonomous vehicles tasked with mapping a set of targets quickly while avoiding collisions with obstacles and other vehicles. This problem could represent a real-world scenario like inspecting a disaster zone or construction site. All necessary constants are defined in a glossary at the end of this document.

Dynamics and Integration

This project will model autonomous vehicles hereafter simply called "agents" with the following simplifying assumptions:

- The effects of buoyancy, lift, and gravity are of secondary importance and may be neglected.
- The agents may propel themselves in any direction in 3D space.
- The agents may be idealized as point masses.
- The agents know the locations of all targets, obstacles, and other agents.

This problem should be modeled in a fixed Cartesian basis. The generic equations for the position, velocity and acceleration in this basis are given in vector form by:

$$\boldsymbol{r} = r_1\boldsymbol{e}_1 + r_2\boldsymbol{e}_2 + r_3\boldsymbol{e}_3 \tag{1}$$

$$\boldsymbol{v} = \dot{\boldsymbol{r}} = \dot{r}_1\boldsymbol{e}_1 + \dot{r}_2\boldsymbol{e}_2 + \dot{r}_3\boldsymbol{e}_3 \tag{2}$$

$$\boldsymbol{a} = \ddot{\boldsymbol{r}} = \ddot{r}_1\boldsymbol{e}_1 + \ddot{r}_2\boldsymbol{e}_2 + \ddot{r}_3\boldsymbol{e}_3 \tag{3}$$

Each agent with a given number i will follow Newton's second law of motion:

$$m_i\boldsymbol{a}_i = \boldsymbol{\Psi}_i^{tot} \tag{4}$$

Where $\boldsymbol{\Psi}_i^{tot}$ is the total force vector acting on the agent, defined by:

$$\underbrace{\boldsymbol{\Psi}_i^{tot}}_{\text{net force}} = \underbrace{\boldsymbol{F}_{p,i}}_{\text{prop. force}} + \underbrace{\boldsymbol{F}_{d,i}}_{\text{drag force}} \tag{5}$$

The components of the force for this general system are defined as:

$$\boldsymbol{F}_{p,i} = F_{p,i}\boldsymbol{n}_i^* \tag{6}$$

$$\boldsymbol{F}_{d,i} = \tfrac{1}{2}\rho_a C_{d,i} A_i \|\boldsymbol{v}_a - \boldsymbol{v}_i\|(\boldsymbol{v}_a - \boldsymbol{v}_i) \tag{7}$$

The unit vector $\boldsymbol{n}_i^*$ will be determined by the control parameters described next.
Use the Semi-Implicit Euler method to integrate equation 4 in time to a final time of t_f:

$$\boldsymbol{v}_i(t+\Delta t) \doteq \boldsymbol{v}_i(t) + \boldsymbol{a}_i(t)\Delta t = \boldsymbol{v}_i(t) + \boldsymbol{\Psi}_i^{tot}(t)\tfrac{\Delta t}{m_i} \tag{8}$$

$$\boldsymbol{r}_i(t+\Delta t) \doteq \boldsymbol{r}_i(t) + \boldsymbol{v}_i(t+\Delta t)\Delta t \tag{9}$$

Objects, Interactions, and Control Parameters

Agent behavior is controlled by $\boldsymbol{n}_i^*$, the direction of the propulsive force given in equation 6. For this assignment, there is **no option to control the magnitude of the thrust**. A given agent interacts with all targets, all other agents, and all obstacles at every time step to determine $\boldsymbol{n}_i^*$.

Targets and obstacles are stationary and positioned randomly throughout the region given by:

$$(|x| \leq 100\ m), \quad (|y| \leq 100\ m), \quad (|z| \leq 10\ m) \tag{10}$$

Initial arrangement of the agents, obstacles, and targets is handled for you in the skeleton code.

Objects interact in the following ways:

- If an agent passes within `agent_sight` of a target, that target is "mapped" and removed from the array of target positions.[1]
- If two agents pass within `crash_range` of each other, both agents "crash" and are removed from the array of agent positions.[2]
- If an agent is within `crash_range` of an obstacle, the agent "crashes." The obstacle remains.
- If an agent leaves the region $(|x| \leq 150\ m), \quad (|y| \leq 150\ m), \quad (|z| \leq 60\ m))$, it "crashes."
- In the unlikely event that an agent is close enough to interact with multiple objects in the same time step, the order of priority is: agent-target, agent-agent, agent-obstacle.

A trial should stop automatically once all targets are mapped or all agents are crashed.

Clearly, the agents should propel themselves away from obstacles and other agents (at least at close range) and toward targets. However, deriving an ideal set of rules to determine their behavior is not practical. Accordingly, we will use a general control law with parameters that can be optimized via a genetic algorithm. The distance between objects should be a key function input, since faraway objects should be treated differently than nearby ones. We will use the Euclidean distance or 2-norm between objects, defined between agent i with position $\boldsymbol{r}_i$ and target j at $\boldsymbol{T}_j$ as:

$$d_{ij}^{mt} \stackrel{\text{def}}{=} \|\boldsymbol{r}_i - \boldsymbol{T}_j\| = \sqrt{(r_{i1} - T_{j1})^2 + (r_{i2} - T_{j2})^2 + (r_{i3} - T_{j3})^2} \tag{11}$$

The unit vector between $\boldsymbol{r}_i$ and $\boldsymbol{T}_j$ is given by:

$$\boldsymbol{n}_{i\to j}^{mt} = \frac{\boldsymbol{T}_j - \boldsymbol{r}_i}{\|\boldsymbol{T}_j - \boldsymbol{r}_i\|} \tag{12}$$

The interaction vector $\hat{\boldsymbol{n}}_{i\to j}^{mt}$ between an agent and a target is defined by the direction of $\boldsymbol{n}_{i\to j}^{mt}$ and a weight given by an exponentially decreasing function[3] of d_{ij}:

$$\hat{\boldsymbol{n}}_{i\to j}^{mt} = (\underbrace{w_{t1} e^{-a_1 d_{ij}^{mt}}}_{attraction} - \underbrace{w_{t2} e^{-a_2 d_{ij}^{mt}}}_{repulsion}) \boldsymbol{n}_{i\to j}^{mt} \tag{13}$$

The total interaction vector between agent i and all targets is the sum of all interaction vectors:

$$\boldsymbol{N}_i^{mt} = \sum_{j=1}^{N_t} \hat{\boldsymbol{n}}_{i\to j}^{mt} \tag{14}$$

The interaction with obstacles and other agents follow the same form with different constants. The interaction between agent i and obstacle j with position $\boldsymbol{O}_j$ is given by:

$$d_{ij}^{mo} \stackrel{\text{def}}{=} \|\boldsymbol{r}_i - \boldsymbol{O}_j\| = \sqrt{(r_{i1} - O_{j1})^2 + (r_{i2} - O_{j2})^2 + (r_{i3} - O_{j3})^2} \tag{15}$$

[1] Be thoughtful about removing or flagging objects. If you naively change the size of an array during a `for` loop over that array's indices, for instance, you will have some problems. Additionally, resizing arrays is slow. You should not waste time iterating over irrelevant objects and they must not influence the remaining agents. The exact way you resolve this issue is up to your creativity. Some strategies will be presented in the help sessions.

[2] You may neglect the edge case where three or more agents collide simultaneously.

[3] Exponentially decreasing functions have several good characteristics for this problem: they are very smooth, decrease rapidly, and do not explode for any value of d_{ij}. In principle, other function types could be used.

$$\boldsymbol{n}_{i\to j}^{mo} = \frac{\boldsymbol{O}_j - \boldsymbol{r}_i}{\|\boldsymbol{O}_j - \boldsymbol{r}_i\|} \tag{16}$$

$$\hat{\boldsymbol{n}}_{i\to j}^{mo} = (w_{o1}e^{-b_1 d_{ij}^{mo}} - w_{o2}e^{-b_2 d_{ij}^{mo}})\boldsymbol{n}_{i\to j}^{mo} \tag{17}$$

$$\boldsymbol{N}_i^{mo} = \sum_{j=1}^{N_o} \hat{\boldsymbol{n}}_{i\to j}^{mo} \tag{18}$$

The interaction between agent i and agent j is given by:

$$d_{ij}^{mm} \stackrel{\text{def}}{=} \|\boldsymbol{r}_i - \boldsymbol{r}_j\| = \sqrt{(r_{i1} - r_{j1})^2 + (r_{i2} - r_{j2})^2 + (r_{i3} - r_{j3})^2} \tag{19}$$

$$\boldsymbol{n}_{i\to j}^{mm} = \frac{\boldsymbol{r}_j - \boldsymbol{r}_i}{\|\boldsymbol{r}_j - \boldsymbol{r}_i\|} \tag{20}$$

$$\hat{\boldsymbol{n}}_{i\to j}^{mm} = (w_{m1}e^{-c_1 d_{ij}^{mm}} - w_{m2}e^{-c_2 d_{ij}^{mm}})\boldsymbol{n}_{i\to j}^{mm} \tag{21}$$

$$\boldsymbol{N}_i^{mm} = \sum_{j=1,\, j\neq i}^{N_m} \hat{\boldsymbol{n}}_{i\to j}^{mm} \tag{22}$$

The total interaction between an agent and the environment is given by a weighted sum of the interactions with each type of object:

$$\boldsymbol{N}_i^{tot} = W_{mt}\boldsymbol{N}_i^{mt} + W_{mo}\boldsymbol{N}_i^{mo} + W_{mm}\boldsymbol{N}_i^{mm} \tag{23}$$

Finally, the propulsive force direction $\boldsymbol{n}_i^*$ is given by:

$$\boldsymbol{n}_i^* = \frac{\boldsymbol{N}_i^{tot}}{\|\boldsymbol{N}_i^{tot}\|} \tag{24}$$

All interaction parameters ($W_{mt}, W_{mo}, W_{mm}, w_{t1}, w_{t2}, w_{o1}, w_{o2}, w_{m1}, w_{m2}, a_1, a_2, b_1, b_2, c_1, c_2$) will be determined by your genetic algorithm.

GENETIC ALGORITHM

A good set of control parameters will enable agents to map targets quickly and completely without crashes. The cost function for comparing the outputs from different parameters is:

$$\Pi = w_1 M^* + w_2 T^* + w_3 L^* \tag{25}$$

$$M^* = \frac{\text{(Unmapped targets)}}{\text{(Total targets)}}, \quad T^* = \frac{\text{(Used time)}}{\text{(Total time)}}, \quad L^* = \frac{\text{(Crashed agents)}}{\text{(Total agents)}} \tag{26}$$

$$w_1 = 70, \quad w_2 = 10, \quad w_3 = 20$$

Note that all terms in the cost function are non-dimensional. The weights reflect the relative importance of each term. The “design string" for this problem contains all 15 undetermined constants:

$$\mathbf{\Lambda}^i = \{\Lambda_1^i, \ldots, \Lambda_N^i\} = \{W_{mt}, W_{mo}, W_{mm}, w_{t1}, w_{t2}, w_{o1}, w_{o2}, w_{m1}, w_{m2}, a_1, a_2, b_1, b_2, c_1, c_2\}^i \tag{27}$$

You should initially assume that the values of all design parameters lie in the interval:

$$0 \leq \Lambda_n \leq 2 \; \forall \; n \tag{28}$$

Scale and offset all randomly-generated design strings to span this interval.

IMPLEMENTATION

We have provided skeleton codes for the simulation (simulation_INCOMPLETE.py) and parameter writing scripts (write_parameters_INCOMPLETE.py). animation.py, swarmga.py, ga_class.py, and test_scripts.ipynb do not need to be modified.

Steps:

0. Fill in the values from the Variable Glossary into the write_parameters_INCOMPLETE.py script. Rename it to “write_parameters.py" and run in your terminal.
1. In simulation_INCOMPLETE.py, fill the values from LAM, the list of design variable values, into their respective members in the class.
2. Based on the distances computed in _compute_distances, build the boolean arrays for mapped targets, drone-obstacle collisions, and drone-drone collisions.
3. Using the positions, determine which drones have exited the domain.
4. Compute Eqn. 13, Eqn. 17, and Eqn. 20.
5. Compute Eqn. 23, Eqn. 6, Eqn. 7, and Eqn. 5.
6. Compute the Semi-Implicit Euler updates to velocity (Eqn. 8) and position (Eqn. 9).
7. Compute Eqn. 26 and Eqn. 25.
8. Update the number of remaining targets and drones
9. The format is laid out, fill in the methods in the proper order.
10. Rename the script to “simulation.py" and run the test_scripts.ipynb notebook.

VARIABLE GLOSSARY

Table 1: Dynamics and Integration Parameters

Symbol	Type	Units	Value	Description
A_i	scalar	m^2	1	agent characteristic area
$C_{d,i}$	scalar	none	.25	agent coefficient of drag
m_i	scalar	kg	10	agent mass
$\boldsymbol{F}_{p,i}$	3×1 vector	N	Eqn. 6	Prop. force vector
$\boldsymbol{n}_i^*$	3×1 unit vector	none	Eqn. 24	Prop. force direction
$F_{p,i}$	scalar	N	200	Prop. force mag.
$\boldsymbol{F}_{d,i}$	3×1 vector	m	Eqn. 7	Drag vector
$\boldsymbol{r}_i$	3×1 vector	m	Eqn. 1	agent i position
$\boldsymbol{v}_i$	3×1 vector	m/s	Eqn. 2	agent i velocity
$\boldsymbol{a}_i$	3×1 vector	m/s^2	Eqn. 3	agent i acceleration
$\boldsymbol{v}_a$	3×1 vector	m/s	$\mathbf{0}$	Air velocity
ρ_a	scalar	kg/m^3	1.225	Air density
Δt	scalar	s	.2	Time step size
t_f	scalar	s	60	Maximum task time

Table 2: Objects and Interactions Parameters

Symbol	Type	Units	Value	Description
`agent_sight`	scalar	m	5	Target mapping distance
`crash_range`	scalar	m	2	agent collision distance
N_m	scalar	none	15	Number of initial agents
N_o	scalar	none	25	Number of obstacles
N_t	scalar	none	100	Number of initial targets
$\boldsymbol{O}_j$	3×1 vector	m	Eqn. 10	Obstacle j position
$\boldsymbol{T}_j$	3×1 vector	m	Eqn. 10	Target j position

Table 3: Genetic Algorithm Parameters

Symbol	Type	Units	Value	Description
`children`	scalar	none	6	Strings generated by breeding
`parents`	scalar	none	6	Surviving strings for breeding
`S`	scalar	none	20	Designs per generation
`G`	scalar	none	100	Total generations
$L*$	scalar	none	Eqn. 26	Fraction of agents lost
M^*	scalar	none	Eqn. 26	Fraction of targets remaining
T^*	scalar	none	Eqn. 26	Fraction of time used
w_1	scalar	none	70	Weight of mapping in net cost
w_2	scalar	none	10	Weight of time usage in net cost
w_3	scalar	none	20	Weight of agent losses in net cost

DELIVERABLES

You should present your work in a concise, typed technical report. The report should be crafted such that a classmate of yours who is not in this class could understand the purpose and methods of this project. **This report should not be longer than necessary** to convey the key information completely and clearly. There is no specific suggested length. You are *encouraged* to collaborate with your classmates. Appropriate forms of collaboration include: discussing conceptual challenges, discussing implementation strategies, and comparing results. Inappropriate forms of collaboration include: directly exchanging code, directly exchanging results, directly exchanging written report content. In short: discuss the problem freely with your classmates but write your own code and your own report.

Your figures should be clearly labeled, plotted with clear markers and legends, and have axes that include units. The report should have the following sections, addressing the following specific questions in addition to any other key information for understanding your work:

Results and Discussion:

1. Provide a convergence plot showing the total cost of the best design, the mean of all parent designs, and the mean of the overall population for each generation.

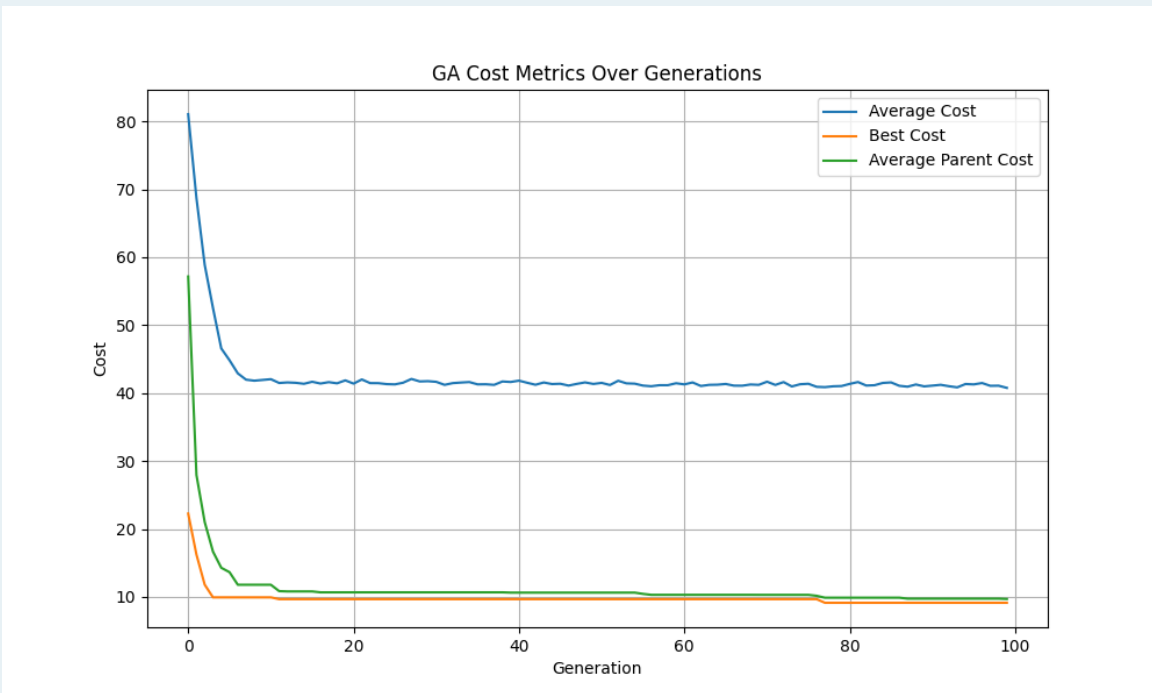

Figure 1: Genetic algorithm convergence history. Shown are cost of the best design (orange), the mean cost of parent designs (green), and the mean cost of the full population (blue).

Observation: The best-cost curve drops rapidly in the first few generations and then flattens, indicating the GA quickly finds a strong region of the search space. The population mean remains significantly higher than the best cost, suggesting that mediocre designs persist even after the best design stabilizes.

2. Provide a plot showing the individual performance components (i.e., plot M^*, T^*, and L^*), for the overall best performer, mean parental population, and overall population. Discuss any important observations.

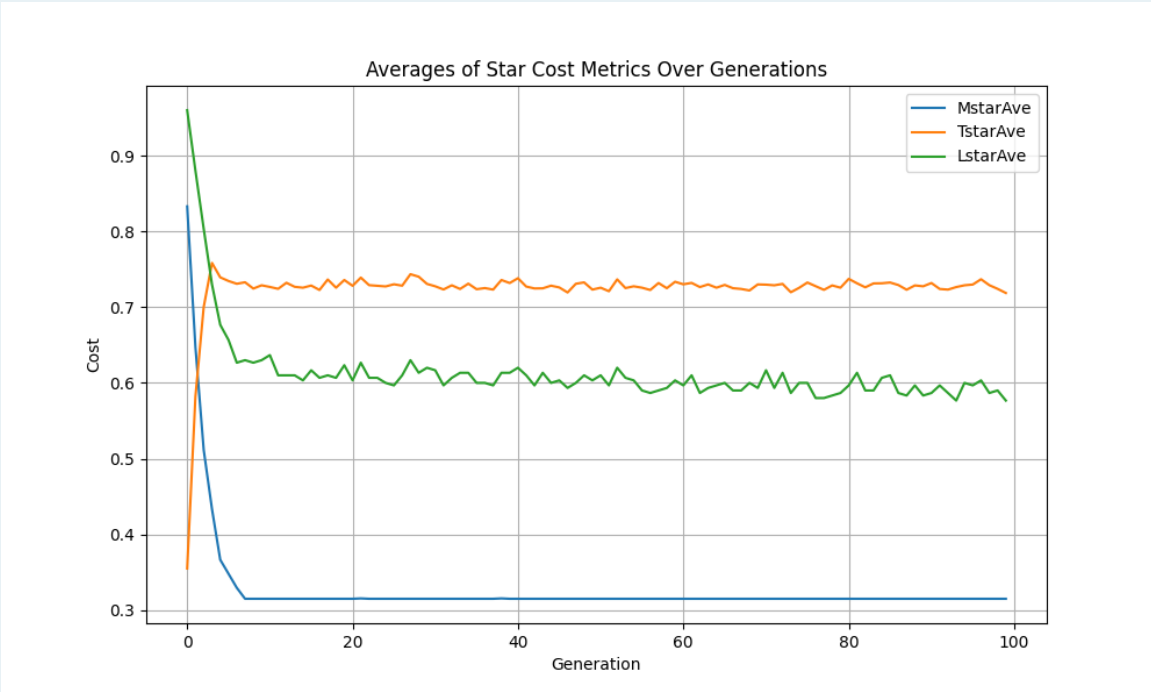

Figure 2: Population-average star metrics (M^*, T^*, L^*) versus generation.

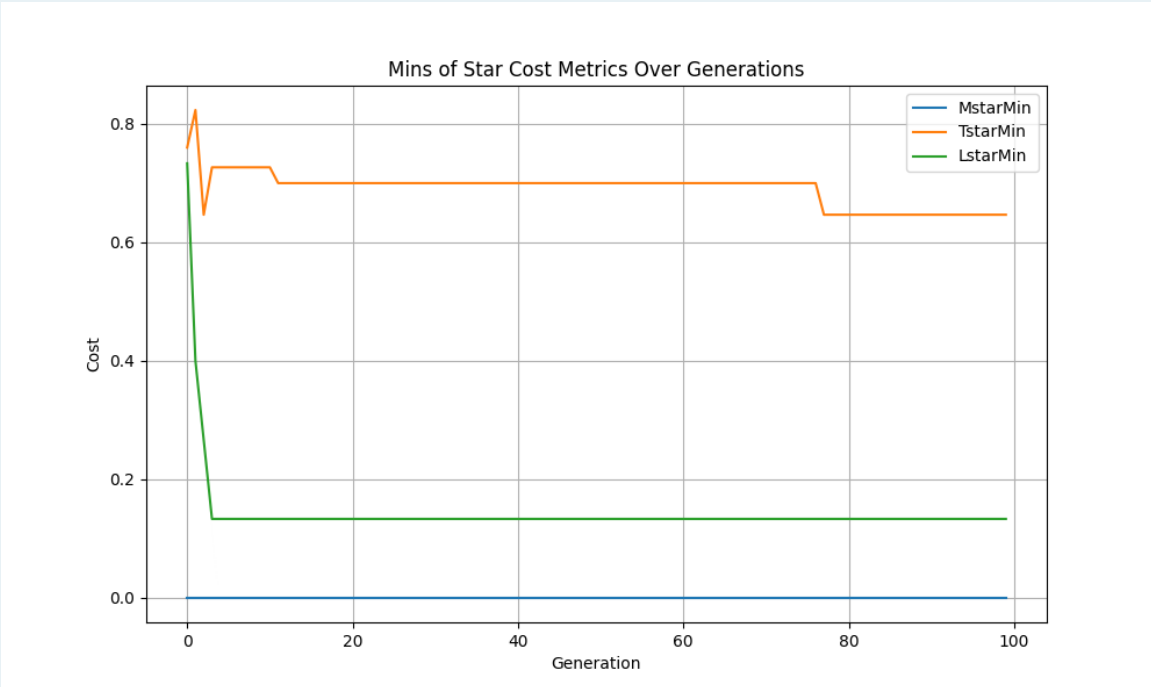

Figure 3: Best-per-generation star metrics (M^*, T^*, L^*) versus generation.

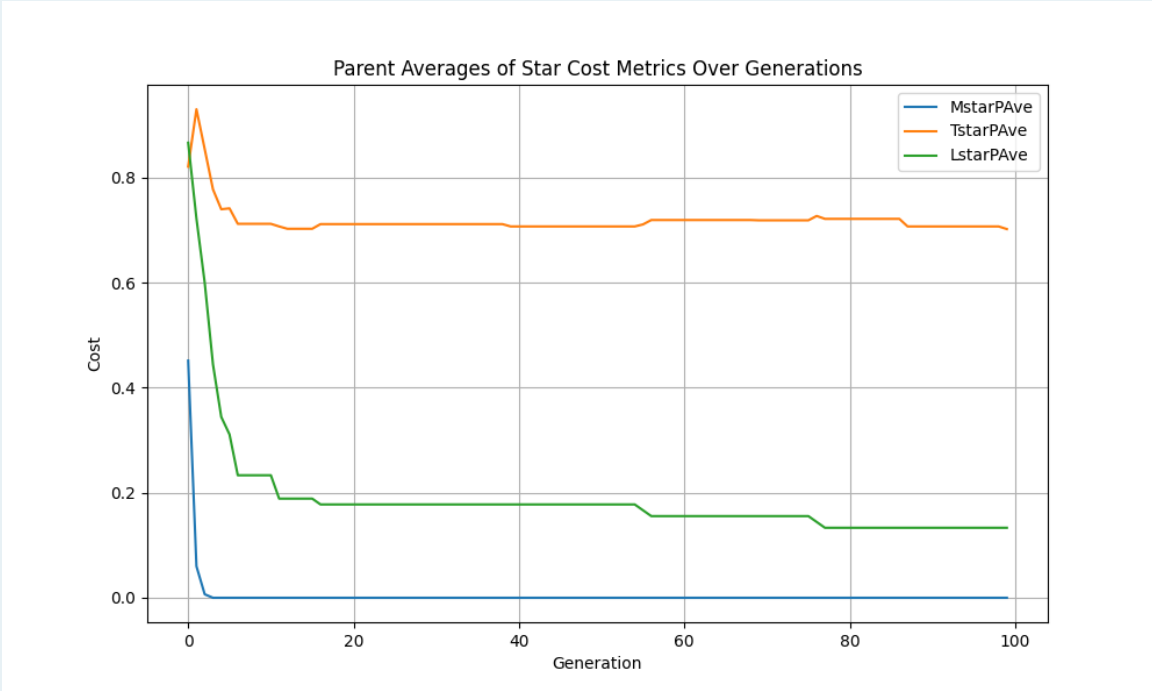

Figure 4: Parent-population average star metrics (M^*, T^*, L^*) versus generation.

Key observations:

- M^* improves quickly early on, meaning the GA learns parameters that allows agents to successfully map targets.
- T^* tends to plateau, suggesting a tradeoff that once mapping becomes reliable, time-to-finish becomes harder to reduce further without increasing losses.
- L^* decreases from early generations and then stabilizes, meaning the GA reduces collision/loss but cannot eliminate them entirely due to environment constraints.

3. Report your best-performing 4 designs in a table similar to the following.

DESIGN	Λ_1	Λ_2	Λ_3	$W_{mt}, W_{mo}, W_{mm}, w_{t1}, w_{t2}, w_{o1}, w_{o2}, w_{m1}, w_{m2}, a_1, a_2, b_1, b_2, c_1, c_2$	Π
1	1.134150	1.887051	0.328122	(1.13 1.88 0.32 1.55 1.33 0.54 0.58 0.71 1.40 0.20 1.44 1.18 1.65 1.06 1.73)	15.633333
2	1.134180	1.885484	0.328960	(1.13 1.89 0.33 1.55 1.33 0.55 0.58 0.72 1.37 0.21 1.44 1.18 1.66 1.07 1.74)	17.133333
3	1.133575	1.889050	0.327642	(1.13 1.89 0.33 1.55 1.33 0.54 0.58 0.72 1.42 0.21 1.45 1.18 1.66 1.07 1.74)	17.533333
4	1.135425	1.881793	0.329494	(1.14 1.88 0.33 1.55 1.33 0.55 0.58 0.72 1.37 0.21 1.44 1.18 1.66 1.07 1.74)	17.833333

Table 4: The top 4 system parameter performers.

4. Discuss the results. How much variation does each parameter have between your top performers? Do any parameters have negative values? Are any values very near zero? Is anything else notable about the results and the behavior they should create?

The top 4 designs are very similar across all 15 parameters, highlighting the strong convergence of the genetic algorithm to a tight region of the search space. Most parameters differ in only the third or fourth decimal. For example, the W_{mo} term from the best design compared to the other designs only differ on the magnitude of 10^{-3} or less. Even the largest variation amongst the parameters (1.42 vs. 1.37 for w_{m2}) is relatively small compared to the allowed design interval $[0, 2]$. This means the GA found a well-defined local optimum.

No parameters have negative values, which is consistent with the bounds $0 \leq \Lambda_n \leq 2$.
No values are extremely close to zero. Among all the top designs, the lowest value is 0.20. This means that every interaction term and all three interaction types (target, obstacle, agent) contribute meaningfully to the performance.
A notable feature from these results is that the obstacle avoidance and agent-agent avoidance parameters are fairly large (many near and above 1.0). This implies that the drones strongly avoid collisions when objects are nearby. The exponential decay parameters are also moderate, which means the influence of objects decrease smoothly with distance rather than disappearing quickly or having strong influence at long range.This creates a very smooth and balanced behavior.

5. Include a series of plots showing how your best design moves the agents. Include 5 approximately evenly-spaced frames from $t = 0$ until the final time for your system. Agents, targets and obstacles should all be clearly shown with distinct marker styles, axes should be labeled and the title should contain the time at which the snapshot occurs. Screenshots are allowed and the frames do not have to be exactly evenly spaced in time.

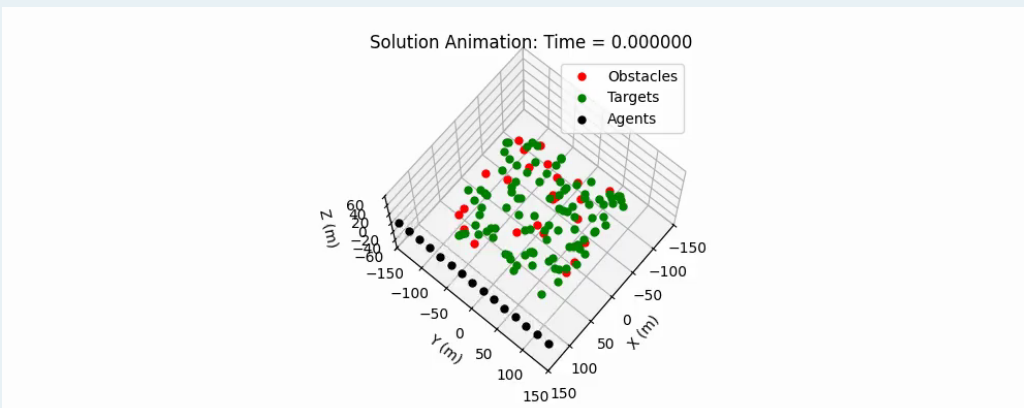

Figure 5: Swarm snapshot at t = 0 s.

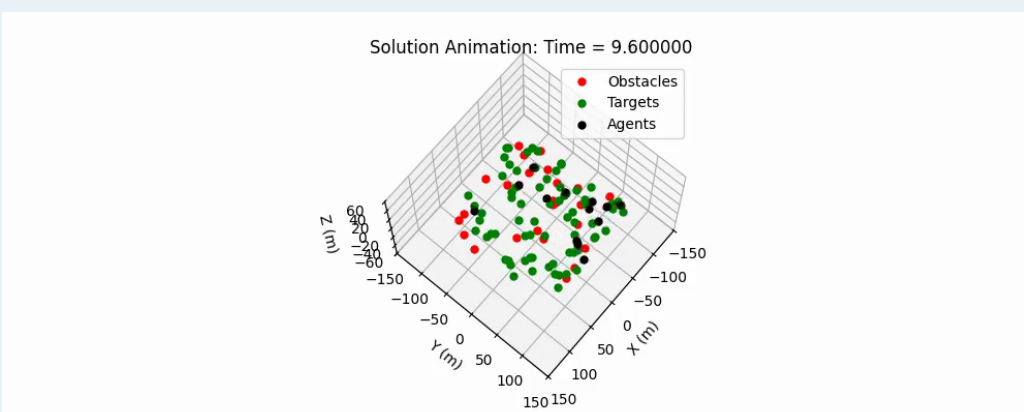

Figure 6: Swarm snapshot at an early time (t = 9.6 s).

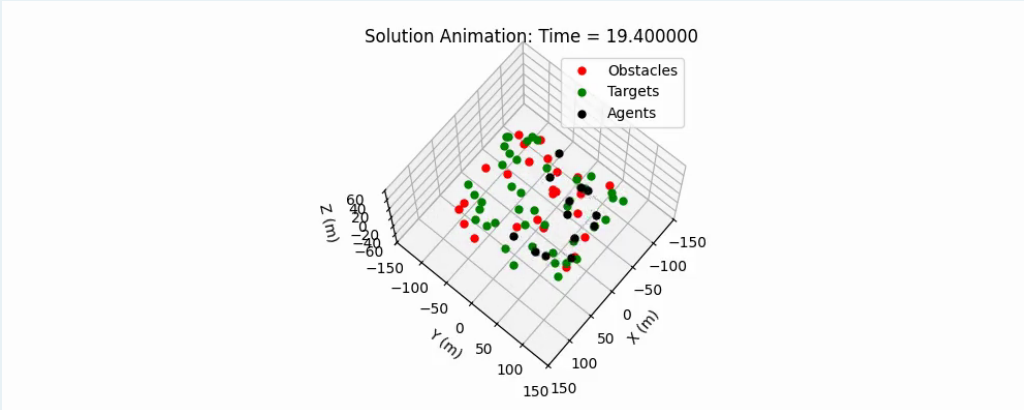

Figure 7: Swarm snapshot at a mid time (t = 19.4 s).

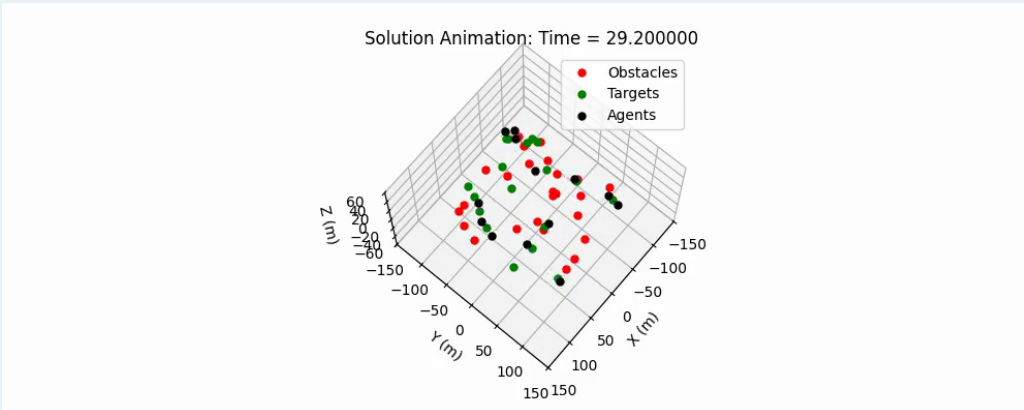

Figure 8: Swarm snapshot at a late time (t = 29.2 s).

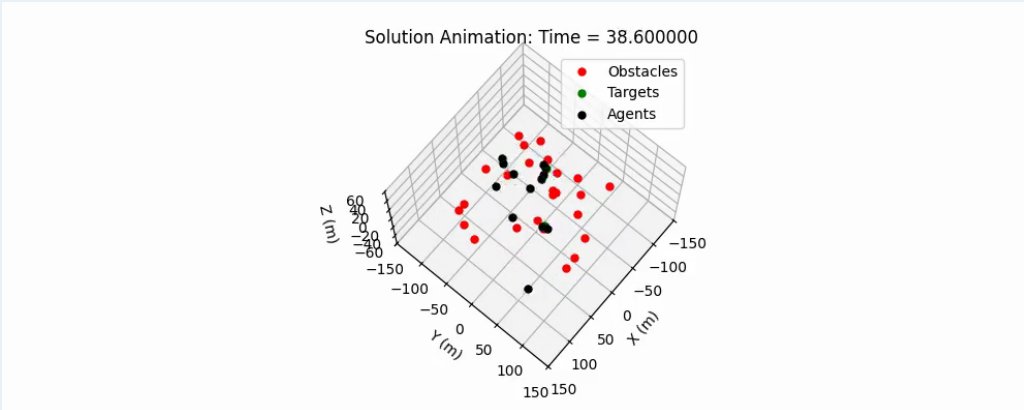

Figure 9: Final swarm snapshot at termination (t = 38.6 s).

To submit your assignment, do BOTH of the following:

1. Submit your PDF copy of your typed report on Gradescope before the end of day (11:59 pm PST).
2. Submit a zip file to Gradescope containing your code (as a .py file) and any supporting files required to run your code.